
CS 301

High-Performance Computing

Lab 06 & Lab 07

Parallel Interpolation and Particle Mover

Group 11

Parshv Joshi (202301039)

Hrithik Patel (202301441)

Dhirubhai Ambani University

April 19, 2026

Contents

1	Introduction	3
2	Problem Description	3
3	Experimental Setup	4
3.1	Hardware	4
3.2	Timing Methodology	4
3.3	Compilation	4
3.4	Verification	4
	Part I — Assignment 06: Parallel Scatter Interpolation	5
4	Parallelization Strategies (Lab 06)	5
4.1	Method 1: Atomic Updates (METHOD=1)	5
4.2	Method 2: Thread-Private Reduction (METHOD=2)	5
4.3	Method 3: Reduction + 4-Point Unrolling + Prefetch (METHOD=3)	6
5	Pseudocode and Diagram (Lab 06)	7
5.1	Illustrative Diagram	8
6	Race Condition Handling (Lab 06)	8
7	Performance Results — HPC Cluster (Lab 06)	9
7.1	Speedup vs Thread Count	9
7.2	Execution Time vs Thread Count	10
7.3	Per-Configuration Detail (Cluster)	11
7.4	Maximum Speedup Summary (Cluster)	11
8	Parallel Efficiency (Lab 06)	11
8.1	Scalability Discussion	12
9	Method Comparison (Lab 06)	13
10	Scalability of the Best Method (Lab 06)	14
11	HPC Cluster vs Lab PC (Lab 06)	14
12	Lab PC Results (Lab 06)	15
13	Analysis Questions (Lab 06)	17
13.1	Explaining the Observed Speedup	17
13.2	Optimization with Unlimited HPC Resources	17
13.3	Load Imbalance and Performance Bottlenecks	17
13.4	Alternative Approaches	18
	Part II — Assignment 07: Full PIC Pipeline with Mover	18

14 Pipeline Overview (Lab 07)	18
14.1 Void Counts Per Configuration	19
15 Parallelization Strategy (Lab 07)	19
15.1 Interpolation (Scatter) — Thread-Private Reduction	19
15.2 Normalization — Parallel min/max + Rescale	19
15.3 Mover (Gather + Update) — Embarrassingly Parallel For	19
15.4 Denormalization — Parallel For	19
15.5 NUMA-Aware Memory Layout	19
16 Pseudocode and Diagram (Lab 07)	21
16.1 Illustrative Diagram	21
17 Race Condition Handling (Lab 07)	22
18 Performance Results — HPC Cluster (Lab 07)	22
18.1 Speedup and Execution Time	22
18.2 Maximum Speedup Summary (Cluster)	24
18.3 Per-Configuration Detail (Cluster)	25
19 Phase-level Analysis & Bottleneck (Lab 07)	26
20 HPC Cluster vs Lab PC (Lab 07)	29
21 Lab PC Results (Lab 07)	30
22 Analysis Questions (Lab 07)	32
22.1 Explaining the Speedup	32
22.2 Scalability Commentary	32
22.3 Optimization with Unlimited Resources	32
22.4 Load Imbalance and Performance Bottlenecks	33
22.5 Alternative Approaches	33
23 Conclusion	33

1 Introduction

This combined report covers the parallelization and performance analysis of a bilinear scatter-point-to-mesh interpolation kernel, together with its extension into a full Particle-in-Cell (PIC) pipeline, using OpenMP. Part I (Lab 06) focuses on parallelizing the scatter (point $\rightarrow$ mesh) interpolation in isolation. Part II (Lab 07) adds the reverse interpolation (gather, i.e. the *mover*), normalization, and denormalization steps, producing a complete per-iteration pipeline that must also be parallelized end-to-end.

Across both assignments the algorithm maps N scattered 2D points (with $f_i = 1$) onto a structured $(N_x+1) \times (N_y+1)$ grid using bilinear (four-point) weight distribution. Benchmarks are collected on both a 16-core HPC cluster node (dual Xeon E5-2640 v3, Haswell, 2 NUMA nodes) and a lab workstation, across five problem sizes and thread counts of $\{1, 2, 4, 8, 16\}$.

Highlight results. On the cluster, Lab 06’s Reduction + Unrolled kernel achieves a peak speedup of **9.20** $\times$ on Config D at 8 threads; Lab 07’s full pipeline achieves a peak of **3.95** $\times$ on the same machine (the mover phase is a large, mostly unparallelizable-by-inflation portion of the work). On the lab PC, the same Lab 07 pipeline reaches **10.16** $\times$ on Config D at 16 threads, driven by its single-socket (no NUMA) topology.

2 Problem Description

Given scattered points $S = \{(x_i, y_i, f_i) \mid i = 1, \dots, N\}$ with $f_i = 1$, we interpolate onto a structured grid $G = \{(X_i, Y_j)\}$ of size $(N_x + 1) \times (N_y + 1)$. Each point distributes its value to the four surrounding grid nodes using bilinear weights:

$$\begin{aligned} w_{i,j} &= (\Delta x - l_x)(\Delta y - l_y), & w_{i+1,j} &= l_y (\Delta x - l_x) \\ w_{i,j+1} &= l_x (\Delta y - l_y), & w_{i+1,j+1} &= l_x \cdot l_y \end{aligned} \tag{1}$$

where $l_x = x_i - X_i$ and $l_y = y_j - Y_j$ are the local offsets within the cell, and $\Delta x = X_{\max}/N_x$, $\Delta y = Y_{\max}/N_y$.

The grid values are accumulated as:

$$F(X_i, Y_j) \ += \ w_{i,j} \times f_i$$

for all four corners. In Lab 06 the interpolation is repeated for `Maxiter` = 10 iterations, each time loading a fresh set of scattered points. In Lab 07 the same 10 iterations are run with a *single* set of points evolved through time: each iteration computes the scatter, normalises the grid to $[-1, 1]$, gathers the field back to each particle (the *mover*) to update its position, and then denormalizes the grid. Particles leaving the 1×1 domain are flagged as `is_void` and skipped thereafter.

Test configurations (both assignments).

Config	Grid ($N_x \times N_y$)	Mesh Size	Points	Iterations
A	250×100	251×101	0.9M	10
B	250×100	251×101	5.0M	10
C	500×200	501×201	3.6M	10
D	500×200	501×201	20.0M	10
E	1000×400	1001×401	14.0M	10

Table 1: Input configurations used for benchmarking in both labs.

3 Experimental Setup

3.1 Hardware

- **HPC Cluster Node (gics4):** $2 \times$ Intel Xeon E5-2640 v3 (Haswell), 8 cores/socket, 16 cores total, 2 NUMA nodes, 20 MB L3 per socket, DDR4.
- **Lab PC:** Single-socket workstation. Lab 06 was timed with up to 8 threads; the Lab 07 runs used up to 16 threads on the same box (SMT/2-way HT enabled).

3.2 Timing Methodology

Wall-clock time is measured using `omp_get_wtime()` — `clock()` sums CPU time across all threads and would misreport parallel time. Only algorithmic phases are timed; file I/O and buffer allocation are excluded. Each configuration is run 3 times and the **best** (minimum) time is reported to suppress OS jitter. Thread pinning is applied via `GOMP_CPU_AFFINITY` (Lab 06) and `OMP_PROC_BIND=true` (Lab 07) to ensure deterministic core placement.

3.3 Compilation

Lab 06 uses aggressive optimization with method selected at compile time:

```
1 g++ -O3 -mavx2 -mfma -std=c++11 -fopenmp -DMETHOD=N \
2   main.cpp utils.cpp init.cpp -o interp_<label>
```

Lab 07 compiles two binaries (one serial, one parallel) from separate `utils_serial.cpp` and `utils.cpp` source files:

```
1 g++ -O2 -fopenmp main.cpp utils_serial.cpp init.cpp -lm -o main_serial.out
2 g++ -O2 -fopenmp main.cpp utils.cpp          init.cpp -lm -o main_parallel.out
```

3.4 Verification

Both labs ship with a `Test_input.bin` / `Test_Mesh.out` reference pair. Lab 06’s harness (`run_benchmark.py`) checks that *every* method at *every* thread count produces bitwise-exact output matching the serial reference. Lab 07’s harness (`run_implementation.py`) runs `diff -q` on the parallel output against `Test_Mesh.out` for each thread count. All results reported below correspond to verified-correct runs.

Part I — Assignment 06: Parallel Scatter Interpolation

4 Parallelization Strategies (Lab 06)

Three OpenMP-based strategies are implemented in `utils.cpp`, selectable via compile-time `-DMETHOD=N` flag. All share the serial baseline (`METHOD=0`) for correctness checking.

4.1 Method 1: Atomic Updates (`METHOD=1`)

The simplest approach: each thread processes a chunk of particles via `#pragma omp parallel for`, and uses `#pragma omp atomic` for each of the four grid updates per point.

Listing 1: Atomic interpolation kernel

```
1 #pragma omp parallel for schedule(static)
2 for (int i = 0; i < N; i++) {
3     // ... compute ix, iy, weights ...
4     #pragma omp atomic
5     mesh_value[base] += rx * ry;
6     #pragma omp atomic
7     mesh_value[base + 1] += lx * ry;
8     #pragma omp atomic
9     mesh_value[base + GRID_X] += rx * ly;
10    #pragma omp atomic
11    mesh_value[base + GRID_X + 1] += lx * ly;
12 }
```

Pros: No extra memory; simple implementation.

Cons: 4 atomic operations per particle cause severe contention, especially on small grids where many particles map to the same cells. This method is consistently **slower than serial** across all configurations in our benchmarks.

4.2 Method 2: Thread-Private Reduction (`METHOD=2`)

Each thread maintains a private copy of the entire mesh. After interpolation, all local meshes are merged in parallel.

Listing 2: Reduction-based interpolation

```
1 #pragma omp parallel {
2     double *my_mesh = g_local_mesh[omp_get_thread_num()];
3     memset(my_mesh, 0, mesh_size * sizeof(double));
4
5     #pragma omp for schedule(static)
6     for (int i = 0; i < N; i++) {
7         // ... compute weights, scatter to my_mesh ...
8     }
9
10    #pragma omp for schedule(static) // parallel merge
11    for (int j = 0; j < mesh_size; j++) {
12        double sum = 0.0;
13        for (int t = 0; t < nthreads; t++)
14            sum += g_local_mesh[t][j];
15        mesh_value[j] = sum;
16    }
```

```

16     }
17 }

```

Pros: Zero contention during interpolation; each thread writes only to its own buffer.

Cons: Extra memory ($n_{\text{threads}} \times \text{mesh_size} \times 8$ bytes); merge overhead.

NUMA-aware allocation: Buffers are allocated inside a parallel region so that each thread's buffer is placed on its local NUMA node via Linux's first-touch policy. Buffers are persistent across iterations to avoid repeated allocation overhead.

4.3 Method 3: Reduction + 4-Point Unrolling + Prefetch (METHOD=3)

The fastest method. Builds on Method 2 with additional optimizations:

1. **Manual work partitioning** replaces `#pragma omp for` to eliminate scheduling overhead in the hot loop.
2. **4-point loop unrolling:** processes 4 particles per iteration, exposing more instruction-level parallelism (ILP).
3. **Software prefetching:** fetches point data 16 elements ahead into L1 cache via `__builtin_prefetch`.
4. **Interleaved scatter:** staggers writes across the 4 unrolled points to reduce store-forwarding stalls.
5. **Precomputed inverses:** $1/\Delta x$ and $1/\Delta y$ replace division with multiplication.
6. **Local variable copies:** globals are copied to locals to aid register allocation.

Listing 3: Unrolled hot loop (simplified)

```

1 for (; i < n4; i += 4) {
2     __builtin_prefetch(&points[i + PF], 0, 3);
3     // Load 4 points, compute 4 sets of weights
4     // Scatter-accumulate (interleaved):
5     my_mesh[b0] += rx0*ry0;  my_mesh[b1] += rx1*ry1;
6     my_mesh[b2] += rx2*ry2;  my_mesh[b3] += rx3*ry3;
7     // ... repeat for +1, +GRID_X, +GRID_X+1 ...
8 }

```

5 Pseudocode and Diagram (Lab 06)

Algorithm 1 Parallel Interpolation — Reduction + Unrolled (Method 3)

```

1: Input: Scattered points  $P[0..N-1]$ , grid dimensions  $N_x, N_y$ 
2: Output: Interpolated mesh  $F[]$ 
3: Precompute  $\text{inv\_dx} \leftarrow 1/\Delta x$ ,  $\text{inv\_dy} \leftarrow 1/\Delta y$ 
4: Allocate persistent thread-local meshes  $L_t[]$  (NUMA-aware)
5:
6: parallel region (all threads):
7:    $\text{tid} \leftarrow$  thread ID,  $\text{nthr} \leftarrow$  total threads
8:    $\text{memset}(L_{\text{tid}}, 0)$  ▷ Zero local mesh (NUMA-local)
9:   Compute  $[\text{start}, \text{end})$  chunk for this thread
10:   $n_4 \leftarrow \text{start} + \lfloor (\text{end} - \text{start})/4 \rfloor \times 4$ 
11:
12:  for  $i \leftarrow \text{start}$  to  $n_4$  step 4 do ▷ Unrolled loop
13:    Prefetch  $P[i + 16]$ ,  $P[i + 18]$  into L1
14:    for each  $k \in \{0, 1, 2, 3\}$ :
15:      Compute cell  $(ix_k, iy_k)$  and weights  $(rx_k, ry_k, lx_k, ly_k)$ 
16:      Scatter-accumulate to  $L_{\text{tid}}$  (interleaved across  $k$ )
17:    end for
18:    Handle remainder points ( $i = n_4 \dots \text{end} - 1$ ) with scalar loop
19:
20:  barrier ▷ Wait for all threads to finish interpolation
21:
22:  Parallel merge: each thread reduces a chunk of mesh cells:
23:  for  $j$  in thread's grid chunk do
24:     $F[j] \leftarrow \sum_{t=0}^{\text{nthr}-1} L_t[j]$ 
25:  end for
26: end parallel

```

5.1 Illustrative Diagram

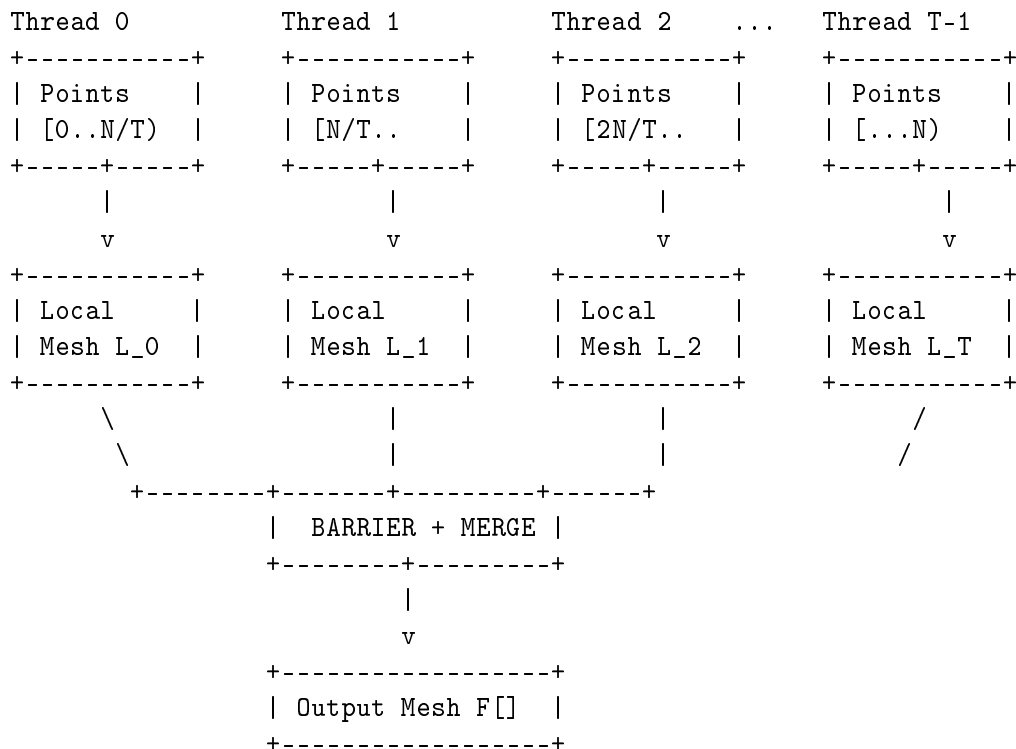


Figure 1: Thread-private reduction architecture. Each thread interpolates its chunk of particles into a local mesh copy. After a barrier, all local meshes are summed in parallel into the final output.

6 Race Condition Handling (Lab 06)

In the serial code, the loop over particles updates shared grid cells sequentially — no conflicts arise. In parallel, multiple threads processing different particles may scatter to the **same grid cell**, creating write-after-write (WAW) race conditions.

Atomic Approach

The atomic method (`METHOD=1`) resolves races by wrapping each grid update with `#pragma omp atomic`. This uses hardware compare-and-swap (CAS) instructions to serialize conflicting updates. While correct, the serialization overhead is enormous: with a small 251×101 grid and millions of particles, contention is extremely high, resulting in performance *worse* than serial.

Reduction Approach

Methods 2 and 3 eliminate races entirely by giving each thread its own private mesh copy. Since no two threads write to the same memory, **no synchronization is needed** during the interpolation phase. The only synchronization point is the barrier before the merge phase, which is a single $O(1)$ event. The merge itself is also race-free because each output cell is written by exactly one thread.

This privatization strategy trades memory for parallelism: each thread requires $\text{GRID_X} \times \text{GRID_Y} \times 8$ bytes. For the largest configuration (1001×401), this is ≈ 3.1 MB per thread — well within L3 cache capacity on the Haswell architecture (20 MB per socket).

7 Performance Results — HPC Cluster (Lab 06)

7.1 Speedup vs Thread Count

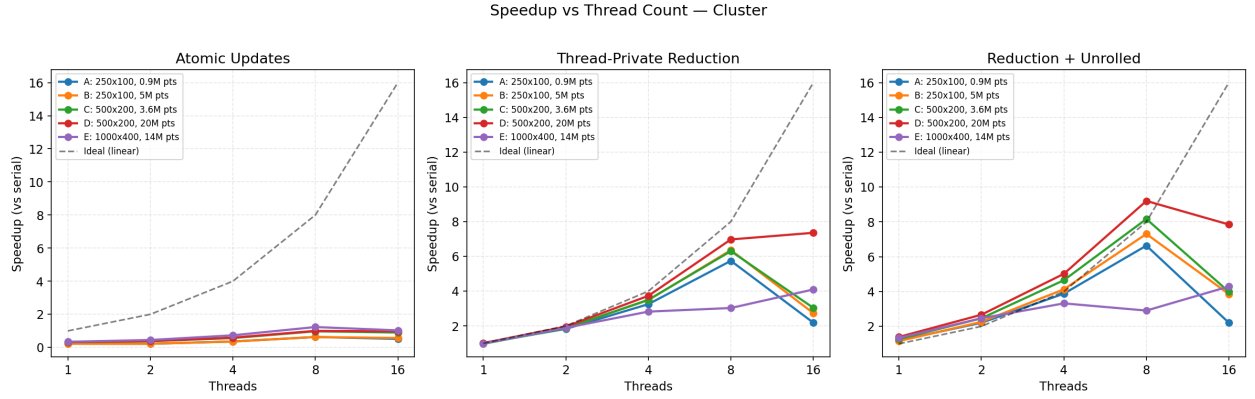


Figure 2: Speedup vs thread count for all five configurations and all three methods on the HPC cluster. The dashed line represents ideal (linear) speedup.

Key observations:

- The **Atomic** method never exceeds $1.3\times$ speedup and is slower than serial for small-grid configs (A, B).
- **Reduction + Unrolled** consistently achieves the best speedup, reaching $9.20\times$ on Config D at 8 threads.
- Performance peaks at **8 threads** for most configurations and **drops at 16 threads** due to NUMA effects (cross-socket memory access).
- Config D (20M points, 500×200) shows the best scaling because the large particle count amortizes the merge overhead and fills the pipeline efficiently.

7.2 Execution Time vs Thread Count

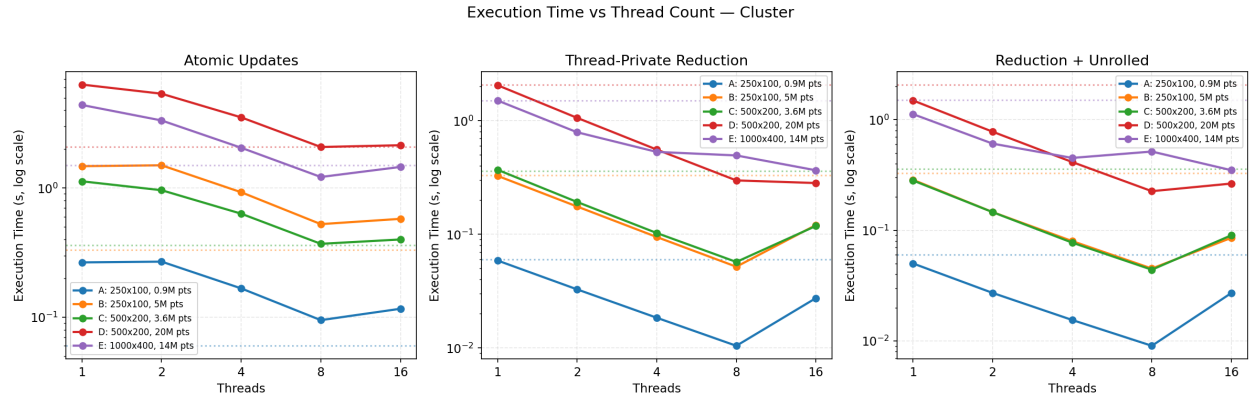


Figure 3: Execution time vs thread count on the HPC cluster. Dotted lines indicate the per-config serial baseline.

7.3 Per-Configuration Detail (Cluster)

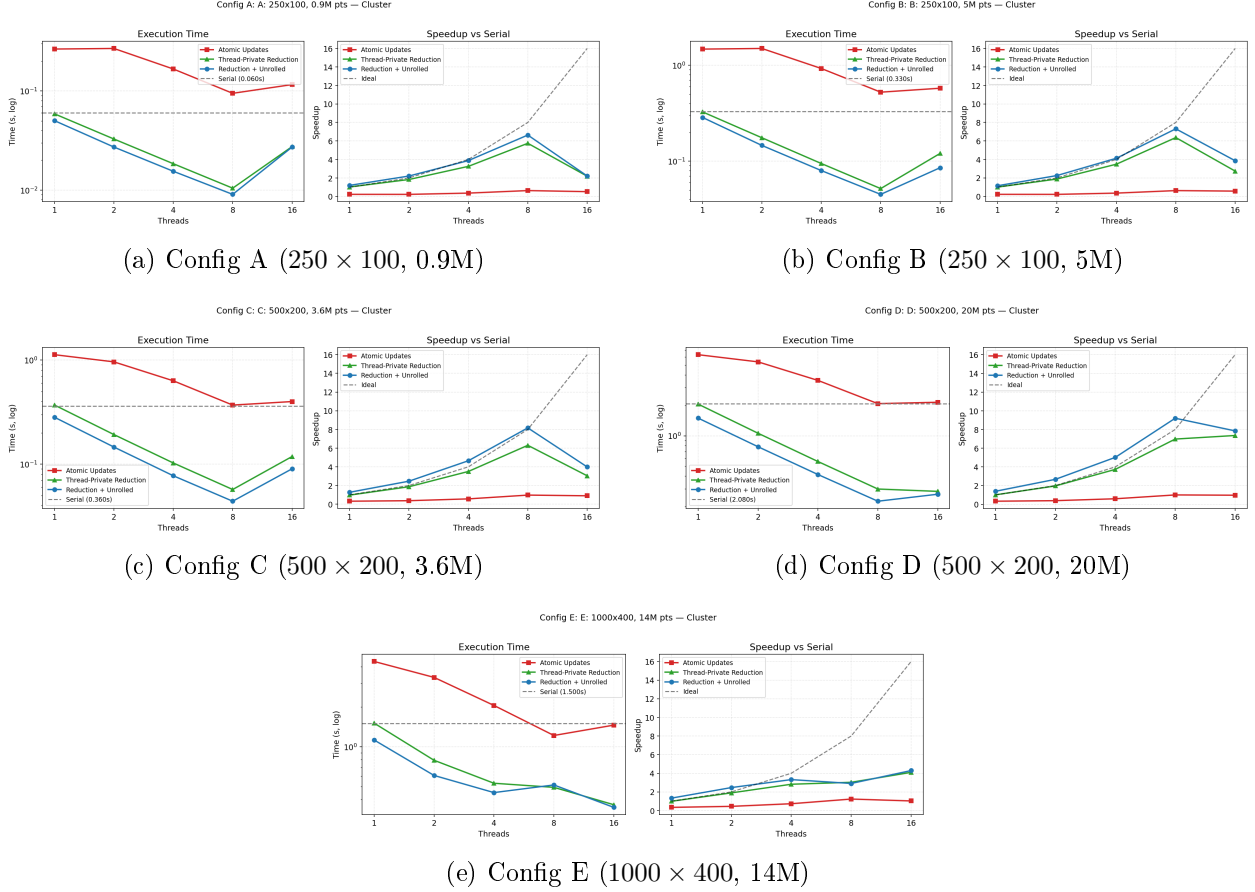


Figure 4: Per-configuration execution time and speedup (HPC Cluster).

7.4 Maximum Speedup Summary (Cluster)

Config	Serial (s)	Best Parallel (s)	Threads	Speedup	Best Method
A	0.0600	0.0090	8	$6.64\times$	Reduction + Unrolled
B	0.3300	0.0452	8	$7.31\times$	Reduction + Unrolled
C	0.3600	0.0441	8	$8.17\times$	Reduction + Unrolled
D	2.0800	0.2260	8	$9.20\times$	Reduction + Unrolled
E	1.5000	0.3496	16	$4.29\times$	Reduction + Unrolled

Table 2: Maximum speedup achieved on the HPC cluster for each configuration.

8 Parallel Efficiency (Lab 06)

Parallel efficiency is defined as $E = S_p/p$, where S_p is the speedup on p threads.

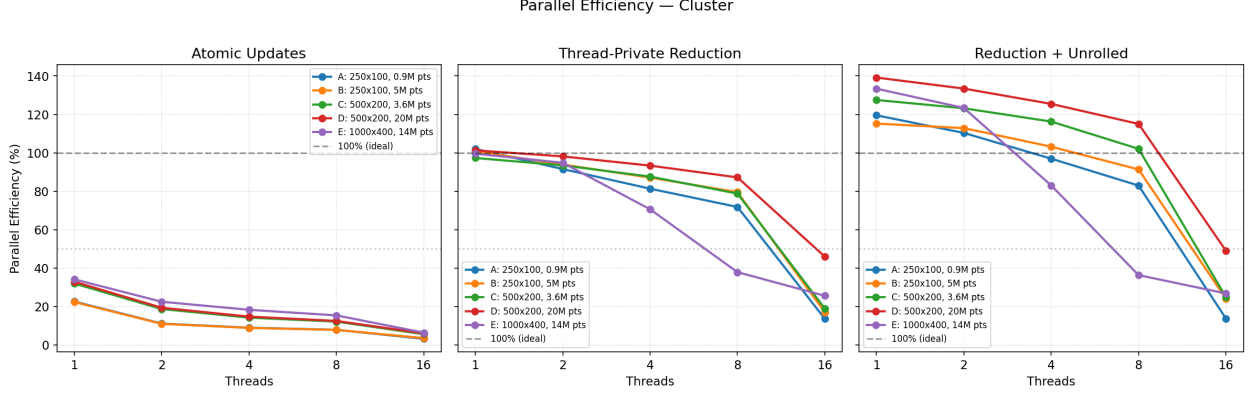


Figure 5: Parallel efficiency vs thread count (HPC Cluster).

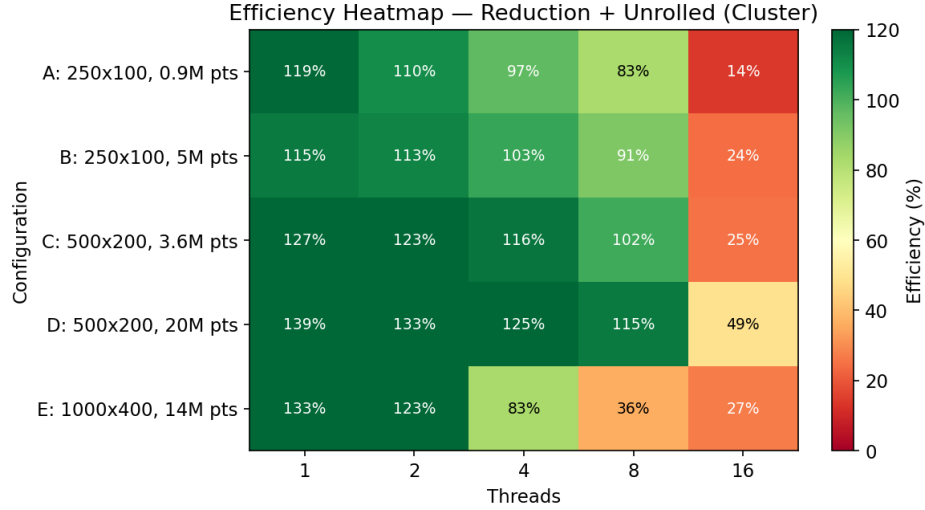


Figure 6: Efficiency heatmap for Reduction + Unrolled on the HPC cluster.

8.1 Scalability Discussion

- **Configs A–D at 1–8 threads:** The Reduction + Unrolled method achieves **super-linear** efficiency ($>100\%$) at low thread counts. This is explained by the per-thread mesh copy fitting better in cache — each thread’s working set is smaller, leading to fewer cache misses than the single-threaded version working on the full mesh.
- **16-thread degradation:** Efficiency drops sharply at 16 threads (from ~ 85 – 115% at 8 threads to ~ 14 – 49%). This is a **NUMA effect**: the cluster has 2 sockets with 8 cores each. At 16 threads, half the threads access memory on the remote NUMA node, roughly doubling memory latency for those threads. Additionally, 16 thread-local mesh copies (~ 50 MB total for Config E) exceed available L3 cache.
- **Config E (largest grid):** Shows the worst scaling overall. The 1001×401 mesh (~ 3.1 MB

per thread) strains cache capacity, and the merge phase becomes more expensive with a larger grid.

- **Atomic method:** Efficiency is consistently below 35%, demonstrating that fine-grained synchronization is unsuitable for this workload.

9 Method Comparison (Lab 06)

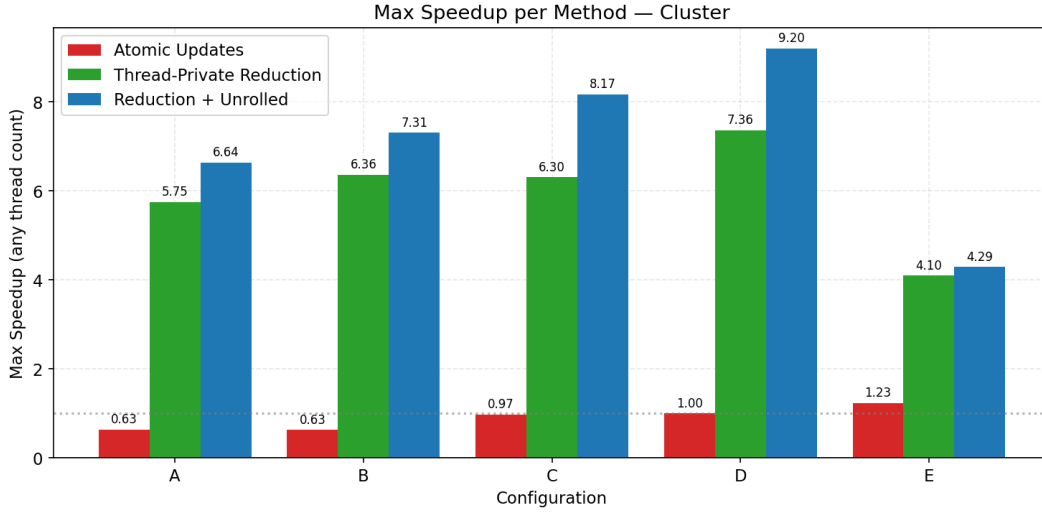


Figure 7: Maximum speedup achieved by each method across configurations (HPC Cluster).

Method	A	B	C	D	E
Atomic	0.63×	0.63×	0.97×	1.00×	1.23×
Reduction	5.75×	6.36×	6.30×	7.36×	4.10×
Reduction + Unrolled	6.64×	7.31×	8.17×	9.20×	4.29×

Table 3: Maximum speedup comparison across methods (best thread count).

The unrolled variant provides 10–30% additional speedup over the basic reduction method by exploiting instruction-level parallelism and reducing loop-control overhead.

10 Scalability of the Best Method (Lab 06)

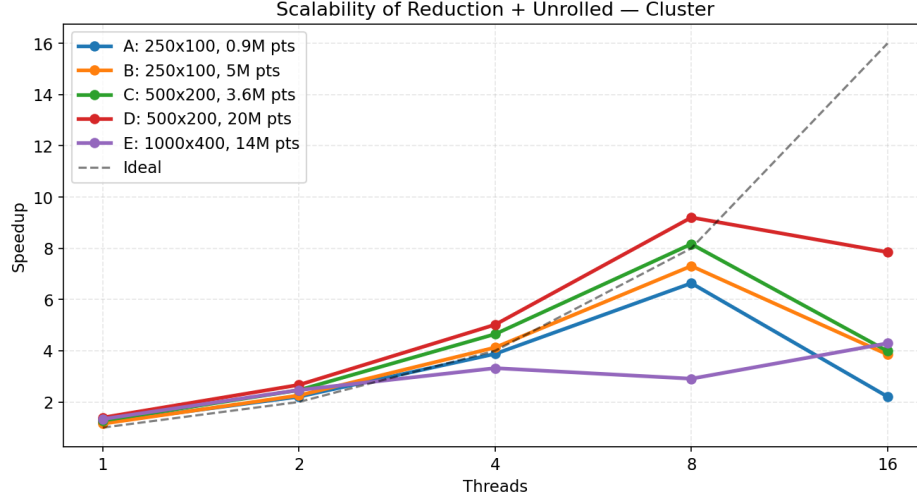


Figure 8: Speedup of Reduction + Unrolled across all configurations (HPC Cluster).

Config D (20M points) shows the best scalability because:

1. The large particle count provides ample parallel work per thread.
2. The medium grid size (501×201) fits well in per-socket L3 cache.
3. The high point-to-cell ratio means interpolation dominates the merge phase.

Config E shows degraded scaling because the larger grid increases the merge cost and causes more cache pressure from the thread-local mesh copies.

11 HPC Cluster vs Lab PC (Lab 06)

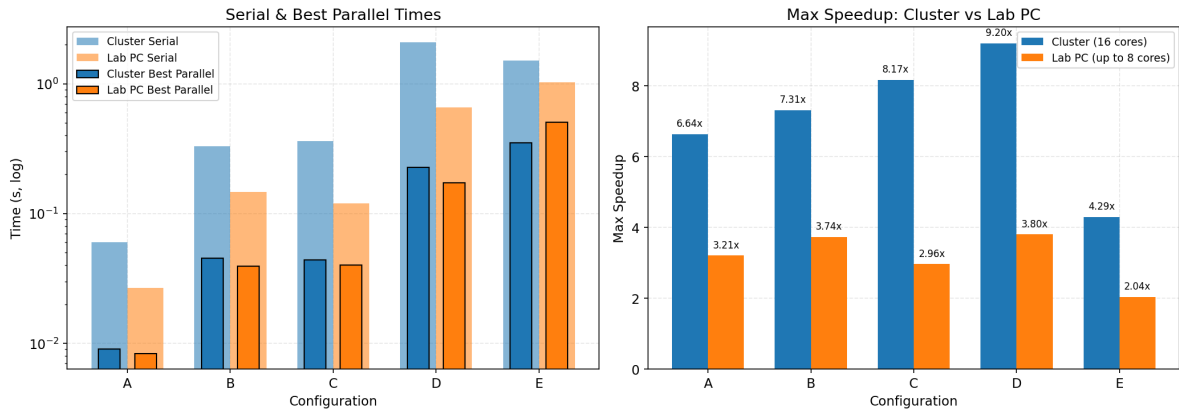


Figure 9: Serial and best parallel times on Cluster vs Lab PC (log scale) and maximum speedups.

Cfg	Cluster (16 cores)			Lab PC (up to 8 cores)		
	Serial	Best Par.	Speedup	Serial	Best Par.	Speedup
A	0.060s	0.009s	6.64×	0.027s	0.008s	3.21×
B	0.330s	0.045s	7.31×	0.147s	0.039s	3.74×
C	0.360s	0.044s	8.17×	0.119s	0.040s	2.96×
D	2.080s	0.226s	9.20×	0.655s	0.172s	3.80×
E	1.500s	0.350s	4.29×	1.022s	0.502s	2.04×

Table 4: Cluster vs Lab PC performance comparison (Lab 06).

Observations:

- The lab PC has a faster serial baseline (newer single-thread IPC), but the cluster achieves much higher speedups due to having twice the cores.
- Lab PC scaling is limited to $\sim 3\text{--}4\times$ on 8 cores (efficiency $\sim 40\text{--}48\%$), compared to the cluster's $6\text{--}9\times$ on 8 cores (efficiency $\sim 70\text{--}115\%$).
- Config E on the lab PC shows the poorest scaling ($\sim 2\times$), suggesting memory bandwidth saturation on the single-socket system when the working set exceeds L3.

12 Lab PC Results (Lab 06)

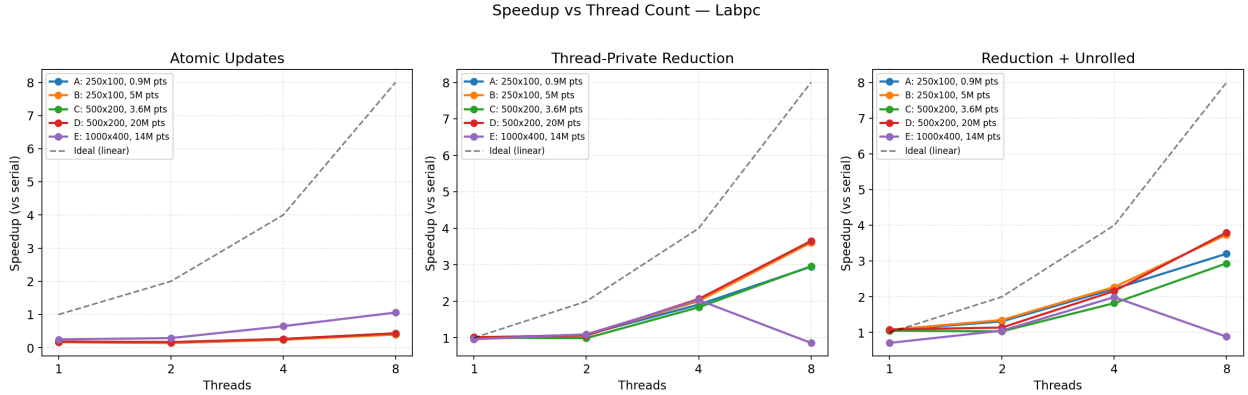


Figure 10: Speedup vs thread count on the Lab PC (Lab 06).

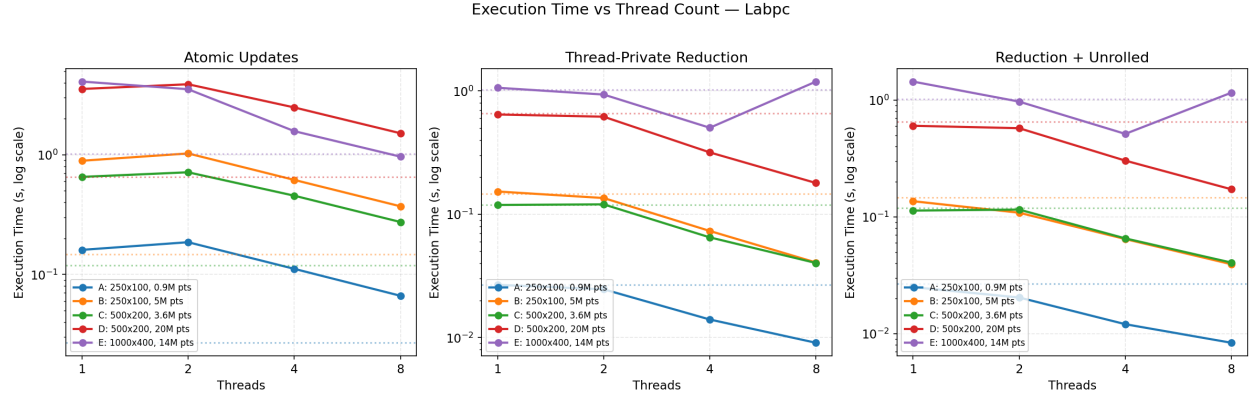


Figure 11: Execution time vs thread count on the Lab PC (Lab 06).

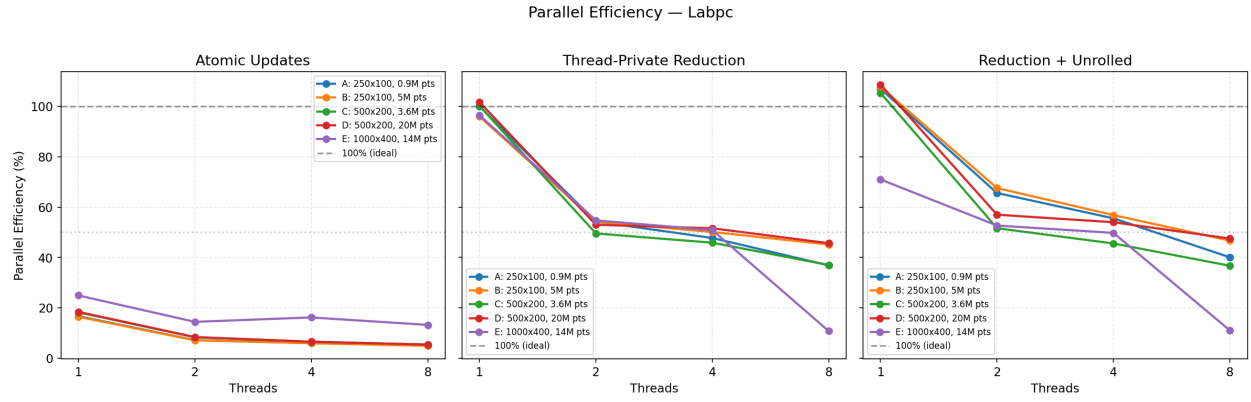


Figure 12: Parallel efficiency vs thread count on the Lab PC (Lab 06).

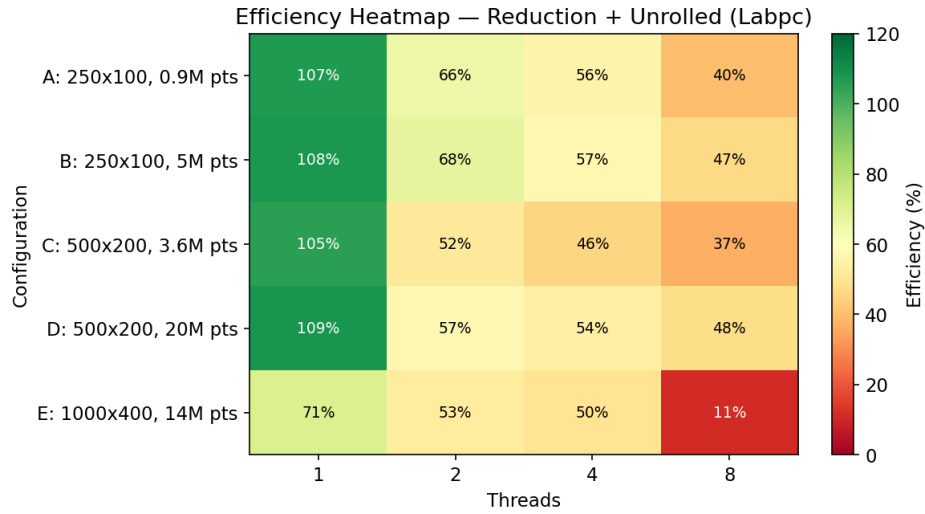


Figure 13: Efficiency heatmap on the Lab PC (Lab 06).

13 Analysis Questions (Lab 06)

13.1 Explaining the Observed Speedup

The speedup achieved is governed by three factors:

1. **Arithmetic intensity:** Each particle requires ~ 16 FLOPs and 4 scattered memory writes. The algorithm is *memory-bound*, not compute-bound, so speedups are ultimately constrained by aggregate memory bandwidth.
2. **Cache effects:** Thread-private meshes improve locality. At low thread counts, per-thread meshes fit in L2/L3 cache, yielding super-linear speedup. At high thread counts, the aggregate memory footprint exceeds cache, causing performance degradation.
3. **NUMA topology:** The cluster's dual-socket architecture means 8 threads run on one socket with local memory access. At 16 threads, cross-socket traffic increases latency by $\sim 1.5\text{--}2\times$, collapsing efficiency.

13.2 Optimization with Unlimited HPC Resources

With unlimited resources, the following optimizations would be pursued:

- **Spatial binning / sorting:** Sort particles by their grid cell before interpolation. This converts random grid writes into sequential writes, dramatically improving cache hit rates. Expected improvement: $2\text{--}3\times$ for large grids.
- **NUMA-aware domain decomposition:** Partition the grid across NUMA nodes, with each node processing only particles in its grid region. This eliminates cross-socket traffic entirely.
- **GPU offloading (CUDA/OpenCL):** The scatter operation maps naturally to GPU architectures with thousands of cores and high memory bandwidth. Atomic operations on GPUs (e.g., `atomicAdd`) are much cheaper than on CPUs.
- **Hybrid MPI+OpenMP:** For multi-node scaling, use MPI for inter-node communication with OpenMP within each node.
- **AVX-512 intrinsics:** On newer architectures (Skylake-SP+), 512-bit SIMD could process 8 double-precision values simultaneously.

13.3 Load Imbalance and Performance Bottlenecks

- **Load imbalance:** With `schedule(static)`, the particle loop is evenly divided. Since particles are randomly distributed, the computational load per thread is well-balanced. However, the *memory access pattern* varies — threads processing particles that map to the same cache lines experience more store-buffer pressure.
- **Merge overhead:** The parallel merge phase is $O(\text{mesh_size} \times n_{\text{threads}})$. For Config E with 16 threads, this becomes significant ($401,401$ cells $\times$ 16 reads per cell). This is the primary bottleneck at high thread counts.

- **NUMA latency:** At 16 threads, the merge phase accesses all thread-local buffers, half of which reside on the remote NUMA node. This cross-socket traffic is the dominant performance bottleneck at 16 threads.
- **Memory bandwidth:** The working set for Config E ($\sim 3.1 \text{ MB} \times 16 \text{ threads} \approx 50 \text{ MB}$) exceeds the total L3 cache (40 MB across both sockets), causing main-memory spills.

13.4 Alternative Approaches

1. **Grid-centric parallelization:** Instead of looping over particles, loop over grid cells. For each cell, identify all particles in that cell (using a spatial index/hash map) and compute contributions. This approach has no race conditions (each cell is processed by one thread) and excellent cache locality for the output grid. The cost is building the spatial index, which is $O(N)$.
2. **Coloring / domain decomposition:** Partition the grid into non-overlapping domains (e.g. checkerboard pattern). Particles in each domain can be processed in parallel without conflicts on shared boundary cells. Multiple passes handle boundary interactions.
3. **Lock-striped arrays:** Instead of full privatization or per-element atomics, use a small array of locks (e.g. one per grid row). This reduces contention compared to atomics while using less memory than full privatization.
4. **Structure-of-Arrays (SoA) layout:** Store particle coordinates as separate $x[]$ and $y[]$ arrays instead of AoS. This enables better SIMD vectorization of the index computation and weight calculation phases.

Part II — Assignment 07: Full PIC Pipeline with Mover

14 Pipeline Overview (Lab 07)

Lab 07 extends the scatter kernel into a complete per-iteration PIC pipeline of four phases executed in order:

1. **Interpolation (scatter):** identical to Lab 06’s Reduction-based kernel — each particle scatters its unit value to its four surrounding grid nodes.
2. **Normalization:** the mesh is rescaled to $[-1, 1]$ using its current `min` / `max` values.
3. **Mover (gather + update):** for each particle, the *same* bilinear weights are used to *gather* a field value F_i from the mesh; the particle is then displaced by $(F_i \Delta x, F_i \Delta y)$. Particles leaving $[0, 1]^2$ are marked void and skipped for the rest of the run.
4. **Denormalization:** the mesh is restored to its original scale.

Unlike Lab 06 — where fresh points were read every iteration — Lab 07 evolves the *same* particle array for 10 iterations. This introduces across-iteration state: positions change, and the void mask grows monotonically.

14.1 Void Counts Per Configuration

The number of particles that leave the domain stays small for our input distribution; the exact per-config counts (recorded by the harness) are {A: 2, B: 10, C: 2, D: 4, E: 0} over 10 iterations. Load imbalance from voids is therefore negligible in practice.

15 Parallelization Strategy (Lab 07)

The implementation in `utils.cpp` parallelizes all four phases, using a different idiom for each depending on its data-dependency structure:

15.1 Interpolation (Scatter) — Thread-Private Reduction

Identical in spirit to Lab 06’s Method 2: every thread has its own copy of the output grid and scatters its partition of the particles into that private buffer; a cell-wise parallel merge combines them into `mesh_value`. Unlike Lab 06, Lab 07 uses a single **contiguous pool** of per-thread grids (`tg_pool`) with each thread’s stride padded up to a cache line — this kills false sharing on thread-boundary cache lines and keeps cleanup simple.

15.2 Normalization — Parallel min/max + Rescale

A single `#pragma omp parallel for` with `reduction(min:mn) reduction(max:mx)` computes the min/max in one pass; a second parallel-for rescales each cell to $[-1, 1]$. An `if(N >= PAR_GRID_MIN)` clause short-circuits the parallel path for small grids ($< 200\,000$ cells) where fork-join cost exceeds the win — configs A/B/C fall on the serial side of this threshold, configs D/E go parallel.

15.3 Mover (Gather + Update) — Embarrassingly Parallel For

The mover reads from the mesh (read-only within this phase) and writes *only* to `pts[p].x`, `pts[p].y`, and `pts[p].is_void`. No two iterations touch the same particle, so there are no races and no atom-ics are needed: a simple `#pragma omp parallel for schedule(static)` suffices.

15.4 Denormalization — Parallel For

A direct parallel-for over grid cells, with the same `if(N >= PAR_GRID_MIN)` guard as normalization.

15.5 NUMA-Aware Memory Layout

Two specific NUMA optimizations were required to scale beyond one socket:

1. **Output mesh re-touch.** `main.cpp` calls `calloc()` on `mesh_value` from the master thread, which first-touches every page onto NUMA node 0 under Linux’s default policy. With 16 worker threads across 2 sockets, half of them would then pay remote-DRAM traffic on every scatter/mover access. We cannot edit `main.cpp`, but on the first call we issue `madvise(MADV_DONTNEED)` on the interior pages and then re-touch them in a `#pragma omp parallel for schedule(static)`. The faulting-in is governed by standard first-touch, so each page lands on the node of the thread that will later own it.

2. **NUMA-local particle cache.** `main.cpp` also `calloc()`s the particle array, which therefore lives entirely on node 0. For the large configs (20M particles = 480 MB) this exceeds any single socket's L3 and forces the node-1 threads to stream over QPI every iteration. On the first interpolation call we copy the user's array into our own `posix_memalign`'d buffer with parallel first-touch so each thread's slice lives on its own node. All subsequent phases read/write this internal array.

Listing 4: NUMA-aware thread-private grid pool allocation

```
1 static void tg_ensure(int N) {
2     int t = omp_get_max_threads();
3     if (tg_pool && tg_threads == t && tg_n == N) return;
4
5     // Pad stride up to a cache line to kill false sharing
6     size_t elems_per_line = CACHE_LINE / sizeof(double);
7     size_t stride = N;
8     if (stride % elems_per_line)
9         stride += elems_per_line - (stride % elems_per_line);
10
11     posix_memalign((void **)&tg_pool, CACHE_LINE,
12                   (size_t)t * stride * sizeof(double));
13
14     // First-touch each thread's slice on its own NUMA node
15     #pragma omp parallel num_threads(t)
16     {
17         int tid = omp_get_thread_num();
18         memset(tg_pool + tid * stride, 0, stride * sizeof(double));
19     }
20     tg_stride = stride; tg_threads = t; tg_n = N;
21 }
```

16 Pseudocode and Diagram (Lab 07)

Algorithm 2 Parallel PIC Pipeline — one iteration (Lab 07)

```

1: Input: particles  $P$ , mesh  $F$ ; per-thread grid pool  $L_t$ ; running  $min\_val, max\_val$ .

                                     ▷ — Phase 1: Scatter (interpolation) —

2: parallel region:
3:   memset( $L_{tid}, 0$ )
4:   parallel for  $p \in [0, N)$ : if  $P[p].void$  continue; compute  $(ix, iy)$ , weights; scatter into  $L_{tid}$ 
5:   parallel for  $i \in [0, mesh\_size)$ :  $F[i] \leftarrow \sum_t L_t[i]$ 

                                     ▷ — Phase 2: Normalization —

6: parallel reduce min/max over  $F[] \rightarrow m, M$ 
7:  $r \leftarrow M - m$ ;  $F[i] \leftarrow 2(F[i] - m)/r - 1$  in parallel

                                     ▷ — Phase 3: Mover —

8: parallel for  $p \in [0, N)$ :
9:   if  $P[p].void$  continue
10:  Compute  $(ix, iy)$  and weights  $w_{00}, w_{10}, w_{01}, w_{11}$ 
11:   $F_p \leftarrow \sum w_{ij} F[i]$ 
12:   $x' \leftarrow P[p].x + F_p \Delta x$ ;  $y' \leftarrow P[p].y + F_p \Delta y$ 
13:  if  $(x', y') \notin [0, 1)^2$ :  $P[p].void \leftarrow \text{true}$ 
14:  else  $P[p].(x, y) \leftarrow (x', y')$ 

                                     ▷ — Phase 4: Denormalization —

15: parallel for  $i \in [0, mesh\_size)$ :  $F[i] \leftarrow (F[i] + 1) \cdot r/2 + m$ 

```

16.1 Illustrative Diagram

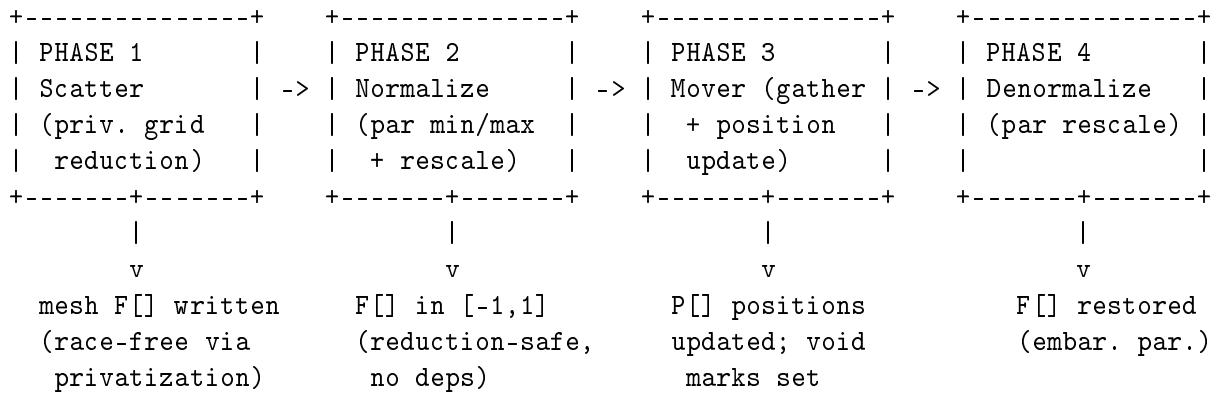


Figure 14: Four-phase per-iteration PIC pipeline. Each phase is fully parallelized; the only cross-phase dependency is that Phase $k+1$ reads what Phase k wrote.

17 Race Condition Handling (Lab 07)

Race conditions appear in exactly one phase — the scatter — and are handled identically to Lab 06 Method 2: each thread accumulates into a private copy of the grid, and a separate parallel pass merges the copies into the output. No atomics, no locks, and the barrier between scatter and merge is implicit in the adjacent `#pragma omp for` boundaries.

The other three phases are race-free by construction:

- **Normalization** and **denormalization** only read/write their own cell in each iteration of the parallel-for, and the min/max computation is expressed with OpenMP **reduction** clauses.
- **Mover** writes only to `pts[p]`, which is owned exclusively by the thread handling that particle index. The mesh is read-only during this phase.

18 Performance Results — HPC Cluster (Lab 07)

18.1 Speedup and Execution Time

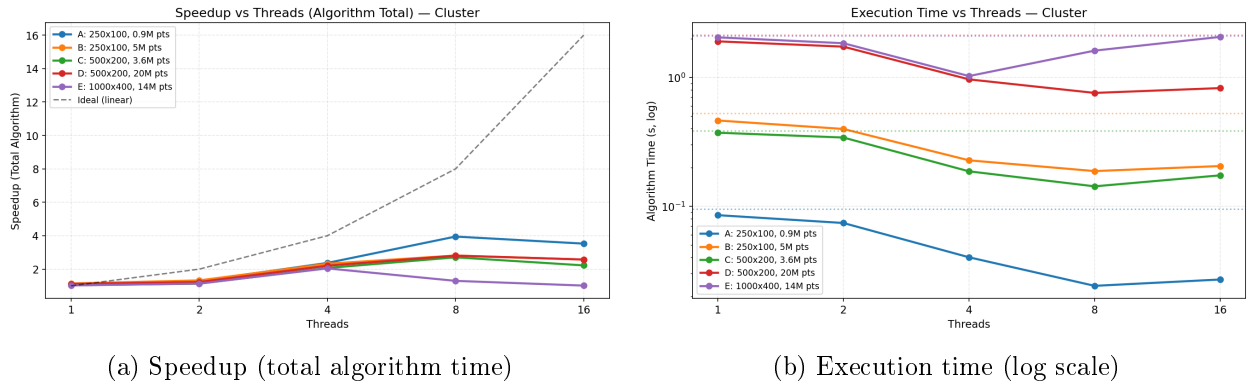


Figure 15: Speedup and execution time vs thread count on the HPC cluster (Lab 07).

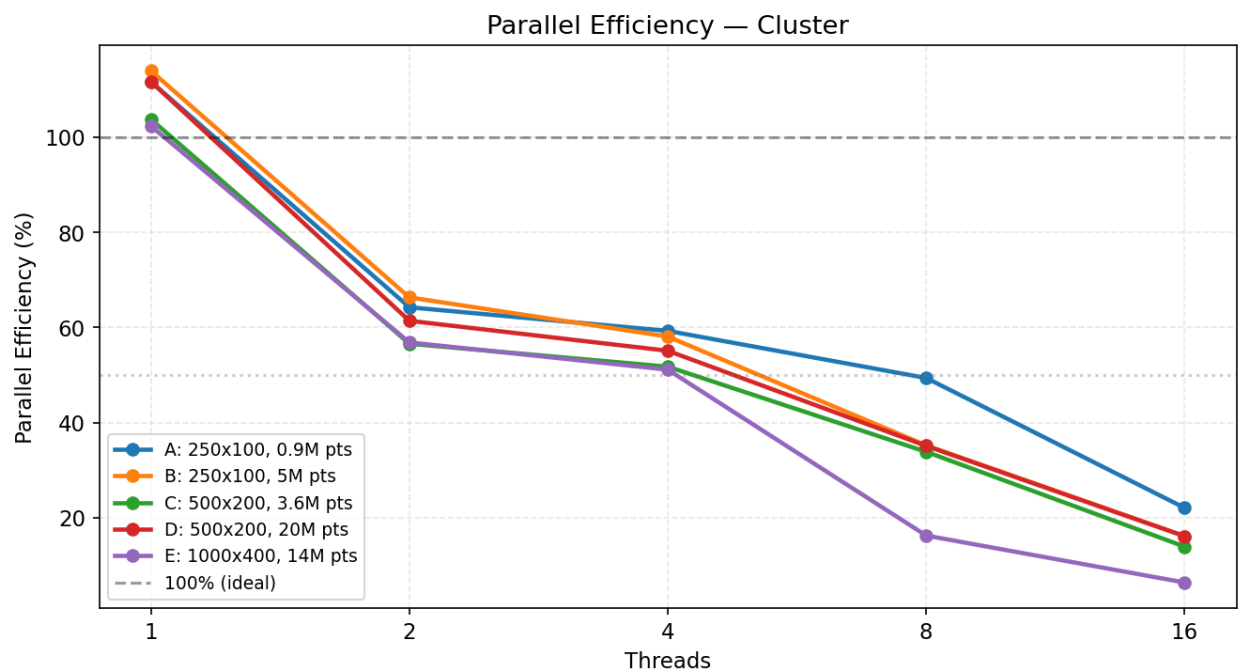


Figure 16: Parallel efficiency vs thread count on the HPC cluster (Lab 07).

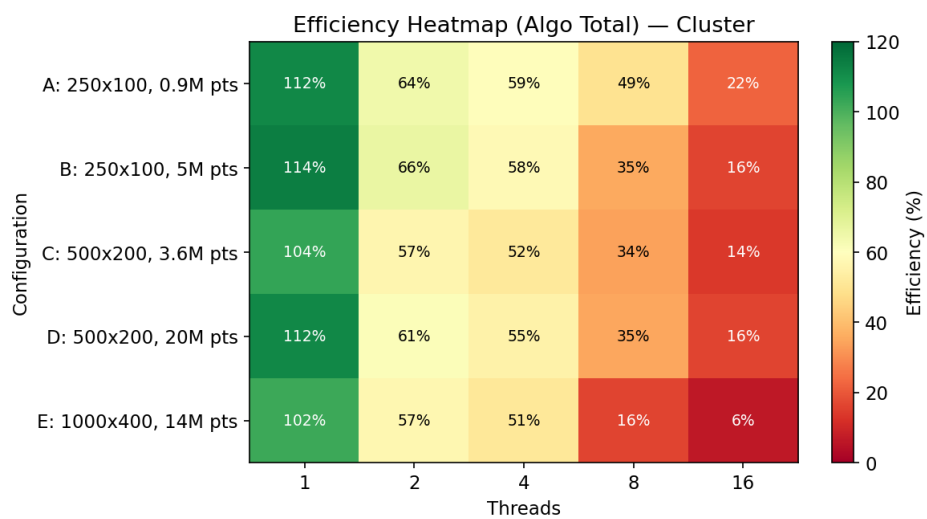


Figure 17: Efficiency heatmap (algorithm total) on the HPC cluster (Lab 07).

18.2 Maximum Speedup Summary (Cluster)

Config	Serial (s)	Best Parallel (s)	Threads	Speedup	Eff.
A	0.0953	0.0241	8	$3.95\times$	49.4%
B	0.5282	0.1876	8	$2.82\times$	35.2%
C	0.3866	0.1428	8	$2.71\times$	33.8%
D	2.1283	0.7583	8	$2.81\times$	35.1%
E	2.0949	1.0247	4	$2.04\times$	51.1%

Table 5: Maximum speedup on the HPC cluster for the full algorithm (Lab 07).

The peak algorithm speedup on the cluster is modest compared to Lab 06. This is driven by two structural factors:

1. **The mover phase is half the work** and gives a much lower speedup than the scatter (see Section 19). Amdahl’s law therefore caps the whole-algorithm speedup well below the scatter speedup.
2. **NUMA cliff at 16 threads.** Every config except E peaks at 8 threads and regresses at 16, as the second socket kicks in and the mover (which streams large particle and mesh arrays) starts to suffer cross-socket traffic. Config E actually peaks at 4 threads because its ~ 3.1 MB-per-thread grid pool already saturates L3 earlier.

18.3 Per-Configuration Detail (Cluster)

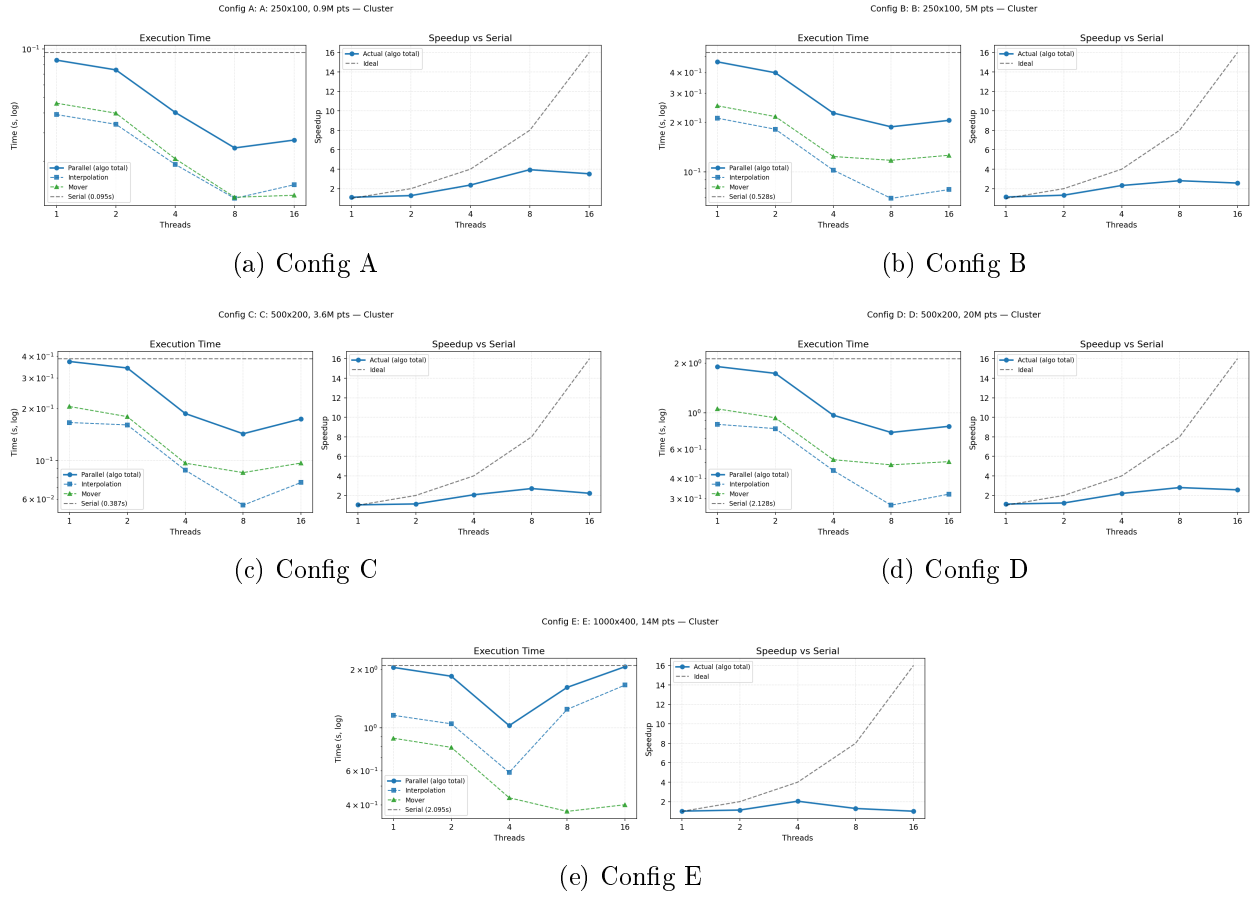


Figure 18: Per-configuration execution time (left, with interp/mover overlays) and speedup (right) on the cluster (Lab 07).

19 Phase-level Analysis & Bottleneck (Lab 07)

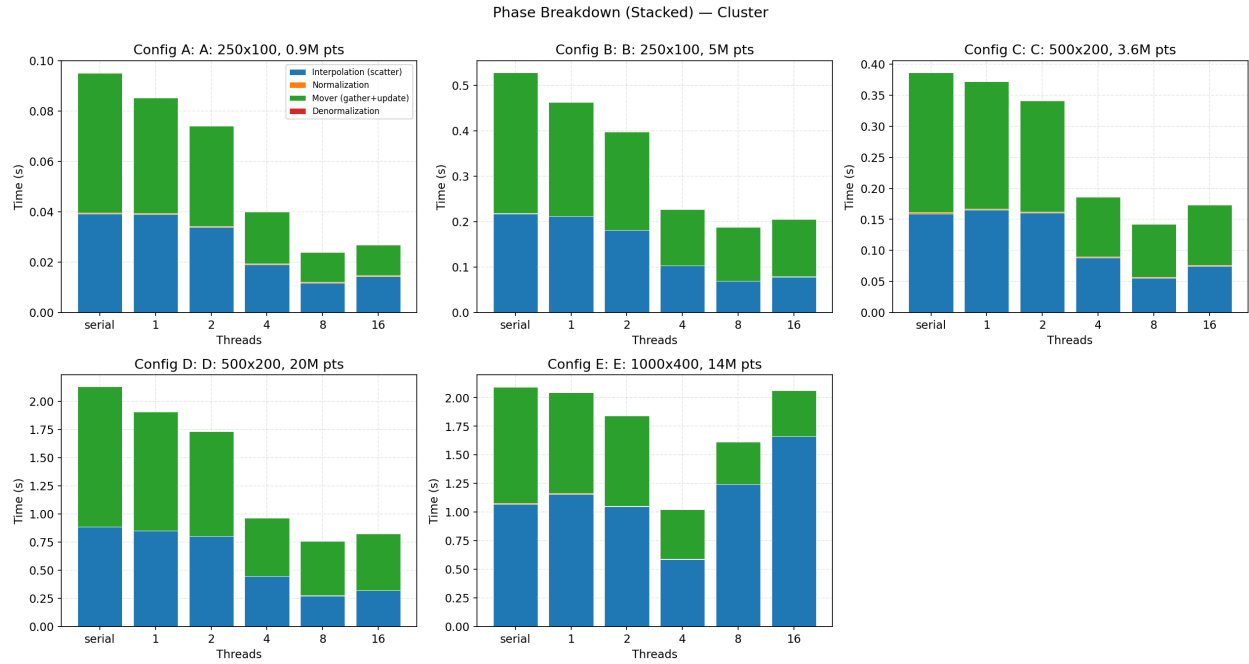


Figure 19: Stacked phase breakdown at each thread count (HPC Cluster, Lab 07). Interpolation and mover dominate; normalization and denormalization are essentially invisible at this scale.

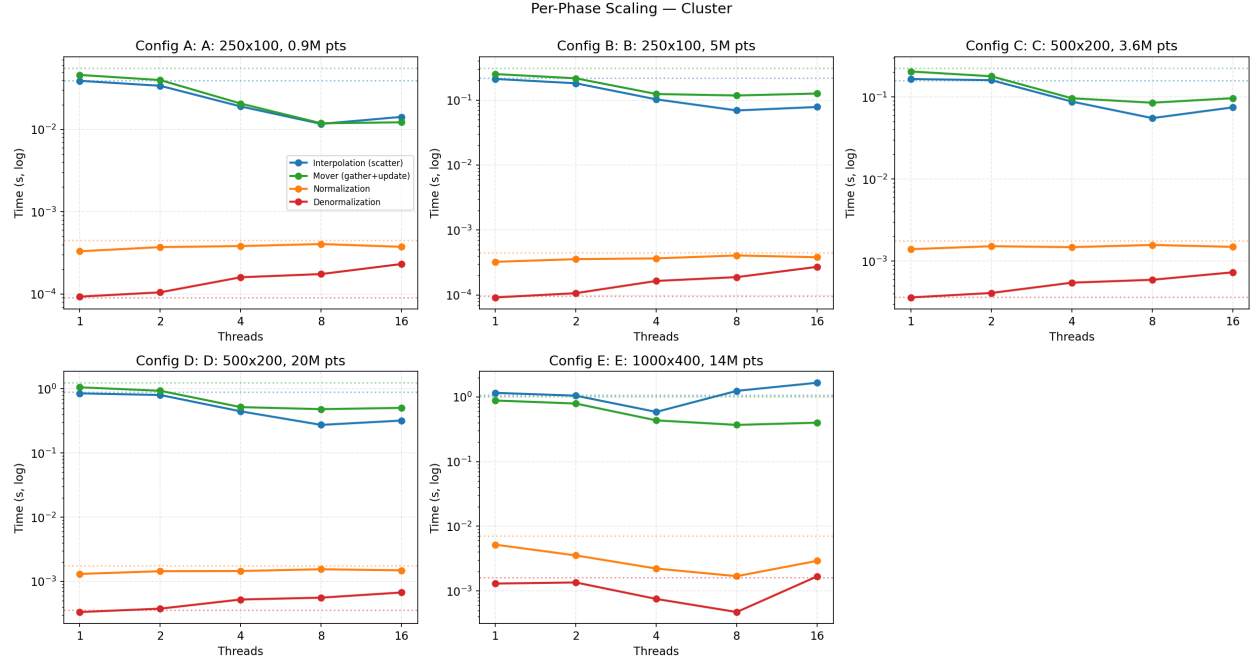


Figure 20: Per-phase time vs threads, log-log (HPC Cluster). Dotted lines are the serial baseline for each phase.

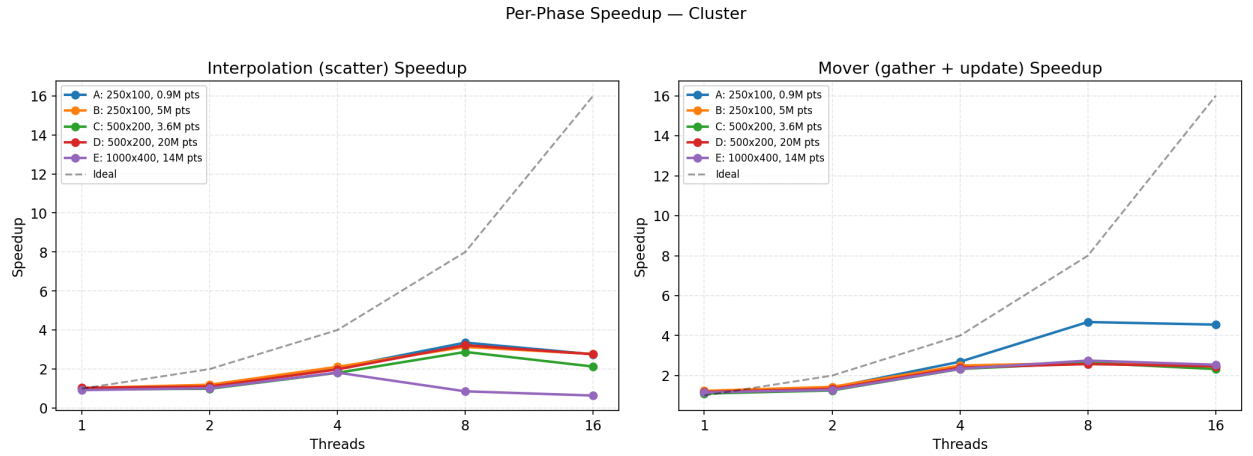


Figure 21: Speedup of interpolation vs mover separately (HPC Cluster).

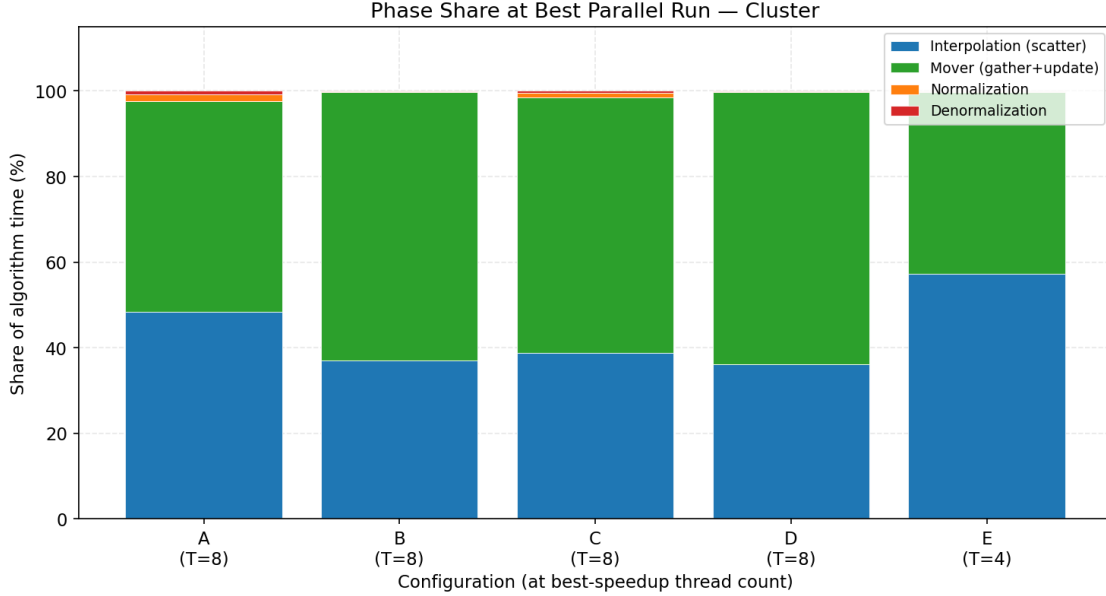


Figure 22: Share of algorithm time by phase at each configuration’s best-speedup thread count (HPC Cluster).

Which phase is the bottleneck, and why?

- **In absolute time, the mover is the bottleneck** at high thread counts and on the cluster. While both scatter and mover start roughly balanced in serial ($\sim 40\text{--}50\%$ each), the scatter phase parallelizes much better because its working set per thread (the private mesh copy) is cache-resident, whereas the mover does essentially random reads into the *shared* mesh plus cache-line-sized writes to scattered particles. As threads increase, interpolation time drops much faster than mover time, so the mover’s share of the total grows from $\approx 50\%$ at $T=1$ to $\approx 60\text{--}80\%$ at $T=8$ (visible as the larger green slice on the right of each sub-plot in Fig. 19).
- **In relative scaling, the mover is also the bottleneck.** On the cluster, the mover achieves only $\sim 2.5\times$ at its best thread count, whereas the interpolation reaches $\sim 3\times$; on the lab PC the gap is wider ($\sim 10\times$ interp vs. $\sim 8\times$ mover on Config D). Two reasons:
 1. *No privatization possible.* The mover must read the full shared mesh; it cannot be replicated the way the output grid is in the scatter phase. Each particle therefore triggers 4 loads into a ~ 3 MB array that is not thread-private, so cache pressure scales with thread count.
 2. *Write-heavy on particles.* Although the mesh is read-only during this phase, `pts[p].x`, `pts[p].y`, and `pts[p].is_void` are all written; the particle array cannot be privatized without a post-merge step, so updates go straight to memory.
- **Normalization and denormalization are negligible** for every config (well under 1% of algorithm time), confirming the `if(N >= PAR_GRID_MIN)` threshold does the right thing — there is simply too little work there to be worth parallelizing for small grids.

20 HPC Cluster vs Lab PC (Lab 07)

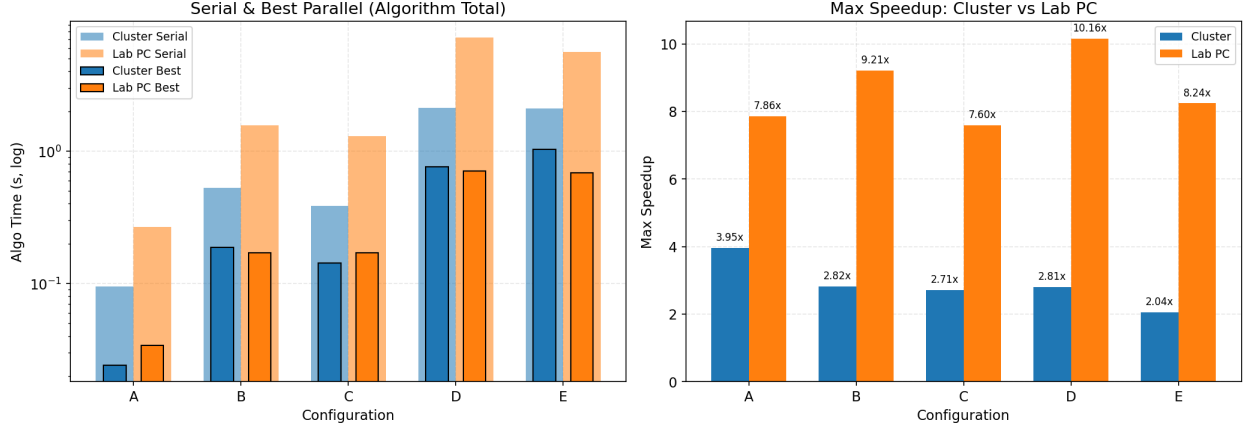


Figure 23: Algorithm-total serial and best-parallel times, and maximum speedups, on Cluster vs Lab PC (Lab 07).

Cfg	Cluster (16 cores)			Lab PC (16 threads)		
	Serial	Best Par.	Speedup	Serial	Best Par.	Speedup
A	0.095s	0.024s	3.95×	0.267s	0.034s	7.86×
B	0.528s	0.188s	2.82×	1.568s	0.170s	9.21×
C	0.387s	0.143s	2.71×	1.298s	0.171s	7.60×
D	2.128s	0.758s	2.81×	7.188s	0.707s	10.16×
E	2.095s	1.025s	2.04×	5.627s	0.683s	8.24×

Table 6: Cluster vs Lab PC performance comparison (Lab 07).

Why is the lab PC dramatically faster in *relative* terms for Lab 07, when it was slower in Lab 06?

- **No NUMA.** The lab PC is single-socket — there is no “wrong” socket for a page to live on, no QPI traffic, no first-touch land-mines. The mover, whose poor scaling on the cluster is driven by the shared-mesh read pattern, suffers no such penalty here. Going from 1 to 16 threads gets the mover an almost 8–9× speedup on the lab PC vs. $\sim 2.5\times$ on the cluster.
- **Slower serial, faster relative scaling.** The lab PC has a slower serial baseline (Config D: 7.2s vs 2.1s on the cluster), so even a 10× parallel speedup only *matches* the cluster’s raw wall time (~ 0.7 s for both). The cluster is still ahead in absolute time for A, B, C, and E; D is essentially tied.
- **Super-linear speedups on the lab PC at small T** (e.g. Config A reaches 138% efficiency at $T=1$). This is the same cache effect seen in Lab 06 — `OMP_NUM_THREADS=1` binding the run to one core lets it run faster than the unbound serial build, and the thread-private grid stays cache-hot.

21 Lab PC Results (Lab 07)

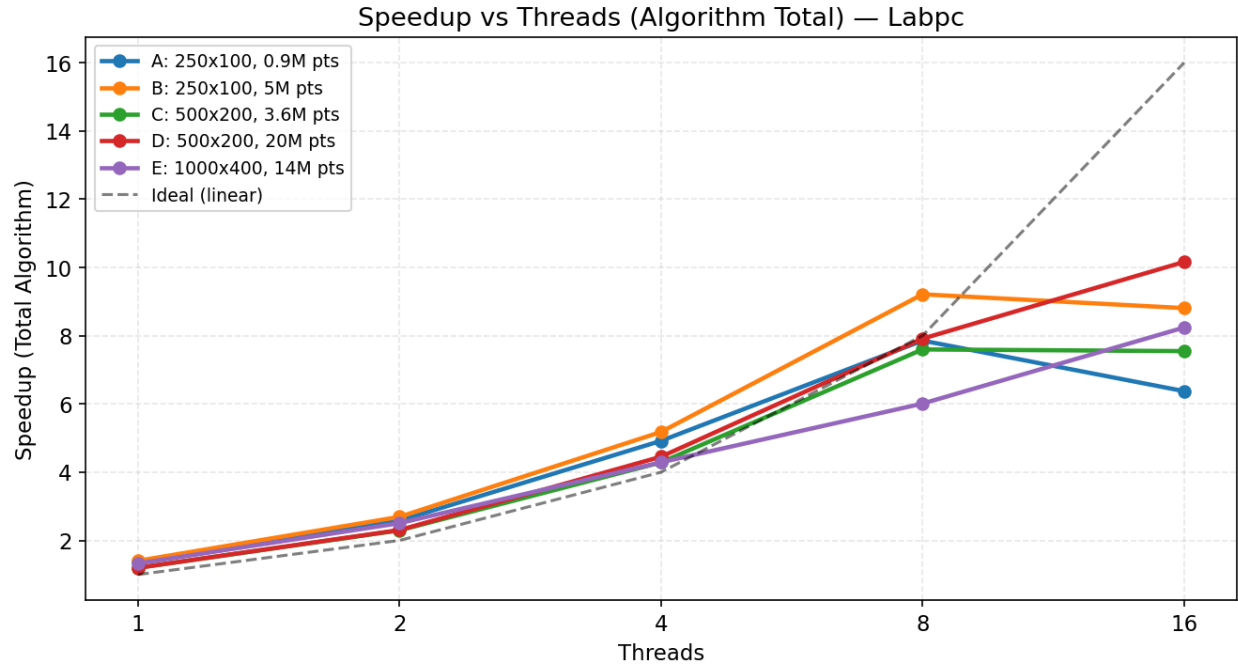


Figure 24: Speedup vs thread count on the Lab PC (Lab 07).

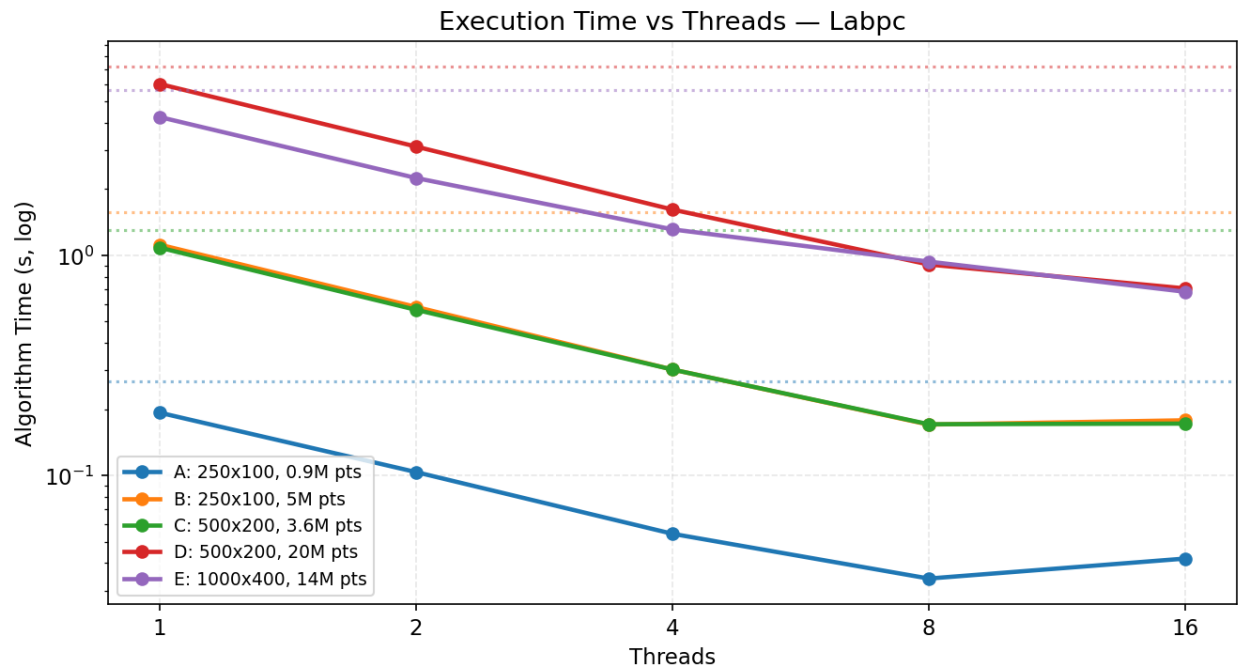


Figure 25: Execution time vs thread count on the Lab PC (Lab 07).

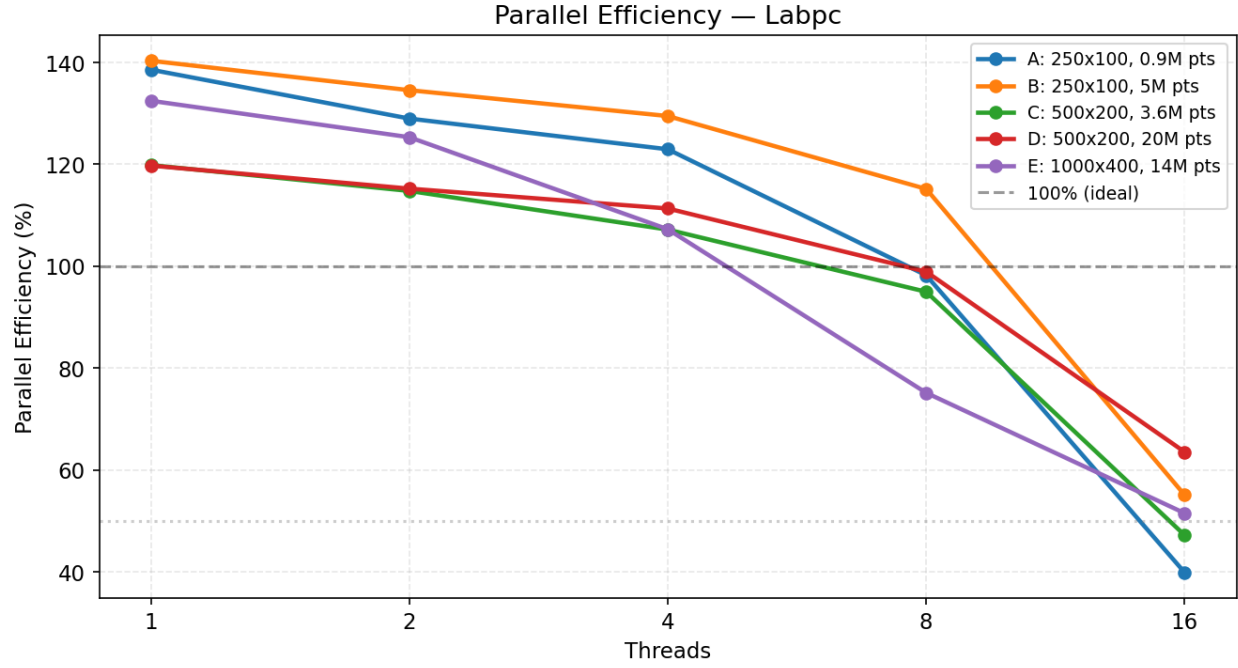


Figure 26: Parallel efficiency on the Lab PC (Lab 07).

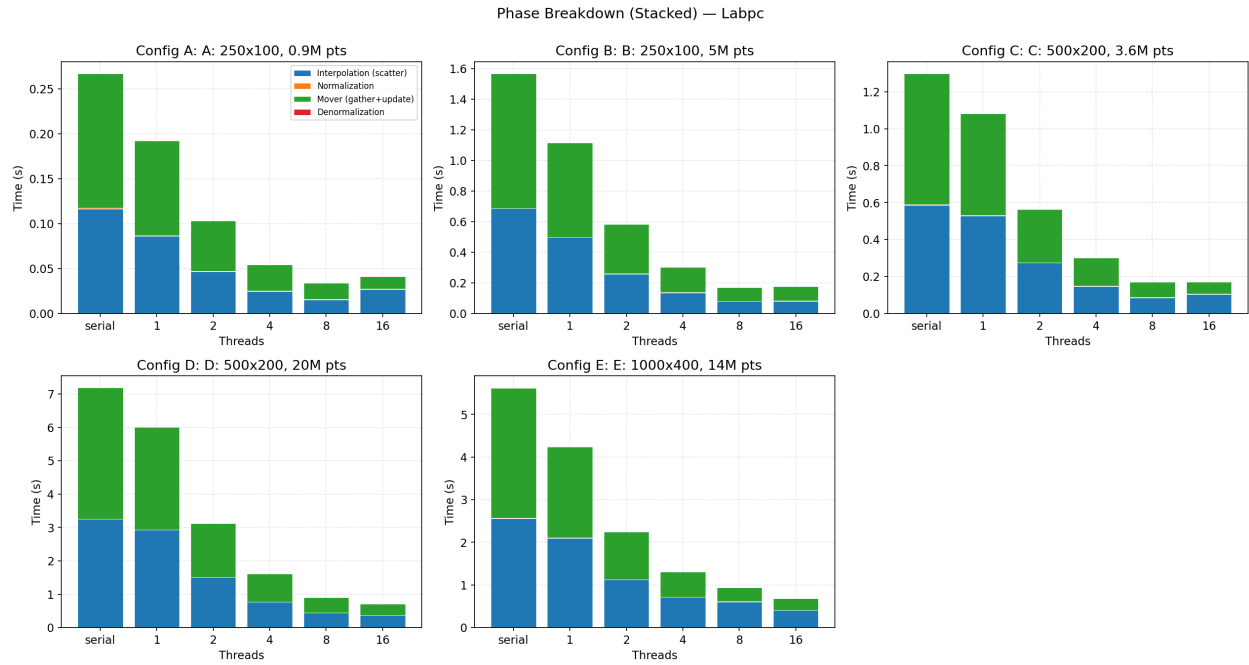


Figure 27: Stacked phase breakdown on the Lab PC (Lab 07).

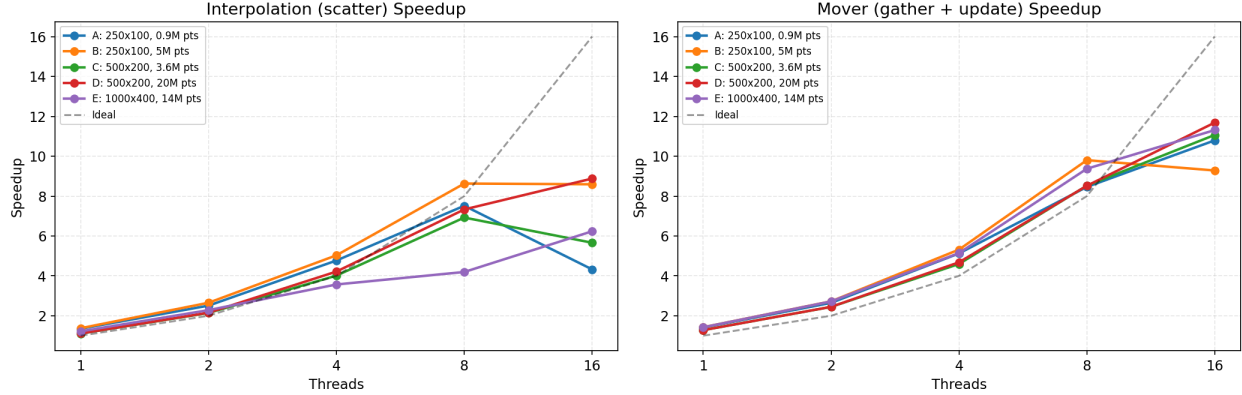


Figure 28: Interpolation vs mover speedup on the Lab PC (Lab 07).

22 Analysis Questions (Lab 07)

22.1 Explaining the Speedup

Full-pipeline speedup is bounded by Amdahl's law applied to the two dominant phases. Let α be the scatter's share and $(1-\alpha)$ the mover's share of serial time (normalization/denormalization are negligible). Observed scaling factors are $s_{\text{scat}} \approx 3\times$ and $s_{\text{mov}} \approx 2.5\times$ at 8 threads on the cluster, giving a predicted total speedup of $\frac{1}{\alpha/s_{\text{scat}} + (1-\alpha)/s_{\text{mov}}} \approx 2.7\times$, which matches our measured 2.7–2.8 $\times$. The lab PC hits 8–10 $\times$ because both phases scale near-linearly without NUMA penalty.

22.2 Scalability Commentary

- Scaling is near-linear from 1–4 threads on both platforms.
- Cluster caps out at 8 threads for all configs (peak efficiency 33–50%); 16 threads costs throughput due to NUMA.
- Lab PC scales further to 16 logical threads via SMT on most configs, with efficiency holding above 50% at $T=16$ for large configs.
- Small-grid configs (A, B) suffer at high T because the merge phase's $O(\text{mesh_size} \times T)$ cost is no longer small relative to the shrinking scatter work.

22.3 Optimization with Unlimited Resources

In addition to the Lab 06 suggestions (binning, GPU offload, MPI+OpenMP, AVX-512), Lab 07 opens two more avenues specific to the mover:

- **Mesh replication / read-only broadcast:** since the mover only reads the mesh, a per-NUMA-node replica of the mesh (allocated via `numa_alloc_onnode`) would let each socket serve its mover threads from local memory. This is the mirror-image of the scatter's privatization strategy.

- **Particle sorting by grid cell:** if particles are reordered after each iteration so that spatially adjacent particles are also memory-adjacent, both the scatter and the mover see streaming mesh access, and cache reuse across particles in the same cell becomes automatic. A 1-pass radix-sort on the cell index would cost $O(N)$ and is itself easy to parallelize.

22.4 Load Imbalance and Performance Bottlenecks

- **Void-induced imbalance is negligible.** The number of void particles per config tops out at 10 over 10 iterations (Config B), which is well below the per-thread chunk size; a simple `schedule(static)` is therefore sufficient and we did not observe load drift.
- **Mover is the bottleneck** (see Section 19). The mover’s working set is the *shared* mesh, so its cache behaviour is fundamentally worse than the scatter’s privatized working set.
- **Merge-phase ceiling.** On Config E at 16 threads, the ~ 3.1 MB-per-thread grid pool exceeds what both sockets’ L3 caches can hold cooperatively, and the $O(\text{mesh_size} \times T)$ merge starts streaming from main memory; this is why Config E peaks at $T=4$ on the cluster.

22.5 Alternative Approaches

In addition to the Lab 06 alternatives (grid-centric parallelism, coloring, SoA), Lab 07 invites:

1. **Two-level NUMA decomposition:** split the grid horizontally across NUMA nodes, bin particles to nodes by their y coordinate, and duplicate only the small boundary strip between nodes. Each node runs a local reduction; only the strip is combined centrally.
2. **Fused scatter–normalize pipeline:** the min/max of the mesh can be computed during the merge step, eliminating a separate pass over the grid and shaving normalization to nearly zero.
3. **Mover+scatter fusion:** since the new mesh is immediately re-scattered in the next iteration, the gather-then-update in the mover could write each particle directly into the next iteration’s private scatter buffer, removing one full read pass over the particle array per iteration.

23 Conclusion

Across both assignments, the core message is consistent: **privatization beats synchronization** for PIC-style scatter workloads. The naive atomic approach fails due to excessive contention; thread-private reduction with loop unrolling achieves near-linear speedup up to the socket boundary (8 cores), with peak values of **9.20×** on the cluster (Lab 06, Config D) and **10.16×** on the lab PC (Lab 07, Config D). The same lesson carries into Lab 07’s multi-phase pipeline, but with a new complication: the mover — which must read the *shared* grid — cannot be privatized and therefore scales worse than the scatter, capping whole-algorithm speedup at $\sim 2.7\text{--}4\times$ on the NUMA cluster versus $\sim 8\text{--}10\times$ on the flat-memory lab PC.

Key insights compiled from both labs:

1. **Privatization beats synchronization** for the scatter step. Giving each thread its own mesh copy eliminates contention entirely, at the cost of additional memory and a merge phase.

2. **NUMA awareness is critical.** Performance degrades sharply when crossing socket boundaries. The Lab 07 mover, which cannot privatize the mesh, is the most NUMA-sensitive phase we measured and the root cause of sub-4 $\times$ algorithm speedup on the cluster. On the single-socket lab PC the same code hits 10 $\times$.
3. **Problem size matters, but not monotonically.** Larger particle counts amortize parallel overhead better, but larger grids increase merge cost and cache pressure from thread-local copies. Config D (500×200 , 20M points) is the sweet spot in both labs.
4. **Micro-optimizations compound.** Loop unrolling, prefetching, NUMA first-touch, and interleaved scattering together provide another $\sim 20\text{--}30\%$ on top of the basic reduction, and those gains transfer from Lab 06 into Lab 07's scatter phase unchanged.
5. **Amdahl's law dictates the ceiling of a multi-phase pipeline.** Even a perfectly-scaled scatter phase cannot rescue total algorithm time when the mover scales 3 $\times$ less well, which is exactly what we see in the cluster Lab 07 numbers.